МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине «Основы машинного обучения»

Тема: «Введение в нейронные сети: построение многослойного перцептрона»

Выполнил:
студент 3 курса
группы АС-67
Швед М. С.

Проверил:
Крощенко А. А.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации.

Общий план:

1. Импорт библиотек и подготовка данных: загрузка датасета, стандартизация (StandardScaler) и кодирование признаков, разделение на обучающую и тестовую выборки, преобразование в тензоры PyTorch;
2. Определение архитектуры нейронной сети: класс, наследуемый от torch.nn.Module, с методами __init__ и forward;
3. Инициализация модели, функции потерь (nn.CrossEntropyLoss для многоклассовой классификации) и оптимизатора Adam (lr = 0.001);
4. Написание цикла обучения: model.train(), прямой проход, расчёт потерь, optimizer.zero_grad(), loss.backward(), optimizer.step();
5. Оценка модели: model.eval(), torch.no_grad(), преобразование логитов в классы (torch.argmax), расчёт метрик с помощью sklearn.metrics.

Используемые инструменты: Python, PyTorch, Pandas, NumPy, Matplotlib, scikit-learn.

Задание по варианту:
Вариант 1. Классификация ирисов

- Iris
- Задача: определить вид ириса (3 класса).
- Архитектура:
  - входной слой;
  - один скрытый слой с 10 нейронами (ReLU);
  - выходной слой с 3 нейронами (Softmax).
- Эксперимент: попробуйте добавить второй скрытый слой с 5 нейронами и сравните точность.

Код:

```
import os


import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix,
ConfusionMatrixDisplay

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
EPOCHS = 1000
LR = 0.001
SEED = 42


# 1. Импорт библиотек и подготовка данных
df = pd.read_csv(os.path.join(BASE_DIR, 'iris.csv'))
print(f"Размер датасета: {df.shape}")
print(df['variety'].value_counts())


X = df.drop('variety', axis=1).values
encoder = LabelEncoder()
y = encoder.fit_transform(df['variety'])          # Setosa -> 0, Versicolor -> 1,
Virginica -> 2
class_names = list(encoder.classes_)


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=SEED, stratify=y)


scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


X_train_t = torch.tensor(X_train, dtype=torch.float32)
X_test_t = torch.tensor(X_test, dtype=torch.float32)
y_train_t = torch.tensor(y_train, dtype=torch.long)
y_test_t = torch.tensor(y_test, dtype=torch.long)
print(f"Обучающая выборка: {X_train_t.shape}, тестовая: {X_test_t.shape}")



# 2. Определение архитектуры нейронной сети
class MLP(nn.Module):
    """4 -> 10 (ReLU) -> 3 (Softmax)"""
    def __init__(self, input_size=4, hidden_size=10, num_classes=3):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden_size)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_size, num_classes)
        self.softmax = nn.Softmax(dim=1)


    def forward(self, x):
        x = self.relu(self.fc1(x))
        return self.fc2(x)  # логиты: nn.CrossEntropyLoss сама применяет Softmax


    def predict_proba(self, x):
        return self.softmax(self.forward(x))



class MLP2(nn.Module):
    """Эксперимент: 4 -> 10 (ReLU) -> 5 (ReLU) -> 3 (Softmax)"""
    def __init__(self, input_size=4, hidden1=10, hidden2=5, num_classes=3):
        super().__init__()
        self.fc1 = nn.Linear(input_size, hidden1)
        self.fc2 = nn.Linear(hidden1, hidden2)
        self.fc3 = nn.Linear(hidden2, num_classes)
        self.relu = nn.ReLU()
        self.softmax = nn.Softmax(dim=1)


    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.relu(self.fc2(x))
        return self.fc3(x)


    def predict_proba(self, x):
```

```
        return self.softmax(self.forward(x))



def train_and_evaluate(model_class, seed=SEED, verbose=True):
    torch.manual_seed(seed)
    # 3. Инициализация модели, функции потерь и оптимизатора
    model = model_class()
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=LR)


    history = {'loss': [], 'test_acc': []}
    # 4. Цикл обучения
    for epoch in range(EPOCHS):
        model.train()
        y_pred = model(X_train_t)
        loss = criterion(y_pred, y_train_t)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()


        model.eval()
        with torch.no_grad():
            acc = (model(X_test_t).argmax(dim=1) == y_test_t).float().mean().item()
        history['loss'].append(loss.item())
        history['test_acc'].append(acc)
        if verbose and (epoch + 1) % 100 == 0:
            print(f"Эпоха {epoch + 1:4d}/{EPOCHS}: loss = {loss.item():.4f}, "
                  f"test accuracy = {acc:.4f}")


    # 5. Оценка модели
    model.eval()
    with torch.no_grad():
        proba = model.predict_proba(X_test_t)
        y_pred_test = torch.argmax(proba, dim=1).numpy()
    acc = accuracy_score(y_test, y_pred_test)
    f1 = f1_score(y_test, y_pred_test, average='macro')
    return model, history, y_pred_test, acc, f1



print("\nМодель 1: 4 -> 10 (ReLU) -> 3 (Softmax)")
model1, hist1, pred1, acc1, f1_1 = train_and_evaluate(MLP)
print(f"Accuracy: {acc1:.4f}, F1-score (macro): {f1_1:.4f}")


print("\nМодель 2: 4 -> 10 (ReLU) -> 5 (ReLU) -> 3 (Softmax)")
model2, hist2, pred2, acc2, f1_2 = train_and_evaluate(MLP2)
print(f"Accuracy: {acc2:.4f}, F1-score (macro): {f1_2:.4f}")


print(f"\nЧисло обучаемых параметров: модель 1 = "
      f"{sum(p.numel() for p in model1.parameters())}, модель 2 = "
      f"{sum(p.numel() for p in model2.parameters())}")


# Эксперимент: сравнение по 10 запускам с разной инициализацией весов
print("\nСравнение по 10 запускам (разные seed):")
for name, cls in [('1 скрытый слой (10)', MLP), ('2 скрытых слоя (10, 5)', MLP2)]:
    accs = [train_and_evaluate(cls, seed=s, verbose=False)[3] for s in range(10)]
    print(f"{name}: accuracy = {np.mean(accs):.4f} ± {np.std(accs):.4f}, "
          f"min = {np.min(accs):.4f}, max = {np.max(accs):.4f}")


# Визуализация
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].plot(hist1['loss'], label='1 скрытый слой (10)')
axes[0].plot(hist2['loss'], label='2 скрытых слоя (10, 5)')
axes[0].set_title('Функция потерь на обучающей выборке')
axes[0].set_xlabel('Эпоха')
axes[0].set_ylabel('CrossEntropyLoss')
```

```
axes[1].plot(hist1['test_acc'], label='1 скрытый слой (10)')
axes[1].plot(hist2['test_acc'], label='2 скрытых слоя (10, 5)')
axes[1].set_title('Точность на тестовой выборке')
axes[1].set_xlabel('Эпоха')
axes[1].set_ylabel('Accuracy')
for ax in axes:
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'training.png'), dpi=150)
plt.show()


fig, axes = plt.subplots(1, 2, figsize=(11, 4.5))
for ax, (title, pred, acc) in zip(axes, [('1 скрытый слой (10)', pred1, acc1),
                                         ('2 скрытых слоя (10, 5)', pred2, acc2)]):
    ConfusionMatrixDisplay(confusion_matrix(y_test, pred),
display_labels=class_names).plot(
        ax=ax, cmap=plt.cm.Blues, colorbar=False)
    ax.set_title(f'{title}: accuracy = {acc:.3f}')
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'confusion_matrices.png'), dpi=150)
plt.show()
```

Результаты выполнения программы:

1. Подготовка данных

Датасет Iris: 150 объектов, 4 числовых признака, 3 сбалансированных класса. Метки классов закодированы LabelEncoder (Setosa → 0, Versicolor → 1, Virginica → 2), признаки стандартизированы StandardScaler. Выборка разделена 80/20 со стратификацией и преобразована в тензоры: признаки - torch.float32, метки - torch.long (этого требует nn.CrossEntropyLoss).

```
Размер датасета: (150, 5)
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
Обучающая выборка: (120, 4), тестовая: (30, 4)
```

2. Архитектура сети

Модель 1 (по варианту): 4 входа → Linear(4, 10) + ReLU → Linear(10, 3) → Softmax; 83 обучаемых параметра. Модель 2 (эксперимент): 4 → Linear(4, 10) + ReLU → Linear(10, 5) + ReLU → Linear(5, 3) → Softmax; 123 параметра. Метод forward возвращает логиты, так как nn.CrossEntropyLoss сама применяет LogSoftmax; слой Softmax используется при получении вероятностей классов (predict_proba). Обучение: оптимизатор Adam, lr = 0,001, 1000 эпох, полный батч.

3. Обучение и оценка

```
Модель 1: 4 -> 10 (ReLU) -> 3 (Softmax)
Эпоха  100/1000: loss = 0.9022, test accuracy = 0.7667
Эпоха  200/1000: loss = 0.5365, test accuracy = 0.8000
Эпоха  300/1000: loss = 0.3595, test accuracy = 0.8333
Эпоха  400/1000: loss = 0.2781, test accuracy = 0.8667
```

```
Эпоха  500/1000: loss = 0.2266, test accuracy = 0.9000
Эпоха  600/1000: loss = 0.1880, test accuracy = 0.9333
Эпоха  700/1000: loss = 0.1578, test accuracy = 0.9667
Эпоха  800/1000: loss = 0.1326, test accuracy = 0.9667
Эпоха  900/1000: loss = 0.1128, test accuracy = 0.9667
Эпоха 1000/1000: loss = 0.0976, test accuracy = 0.9667
Accuracy: 0.9667, F1-score (macro): 0.9666
```

```
Модель 2: 4 -> 10 (ReLU) -> 5 (ReLU) -> 3 (Softmax)
Эпоха  100/1000: loss = 0.8479, test accuracy = 0.7667
Эпоха  200/1000: loss = 0.4597, test accuracy = 0.7667
Эпоха  300/1000: loss = 0.2602, test accuracy = 0.9000
Эпоха  400/1000: loss = 0.1561, test accuracy = 0.9333
Эпоха  500/1000: loss = 0.0986, test accuracy = 0.9667
Эпоха  600/1000: loss = 0.0714, test accuracy = 0.9667
Эпоха  700/1000: loss = 0.0578, test accuracy = 0.9667
Эпоха  800/1000: loss = 0.0504, test accuracy = 0.9667
Эпоха  900/1000: loss = 0.0460, test accuracy = 0.9667
Эпоха 1000/1000: loss = 0.0433, test accuracy = 0.9667
Accuracy: 0.9667, F1-score (macro): 0.9666
```

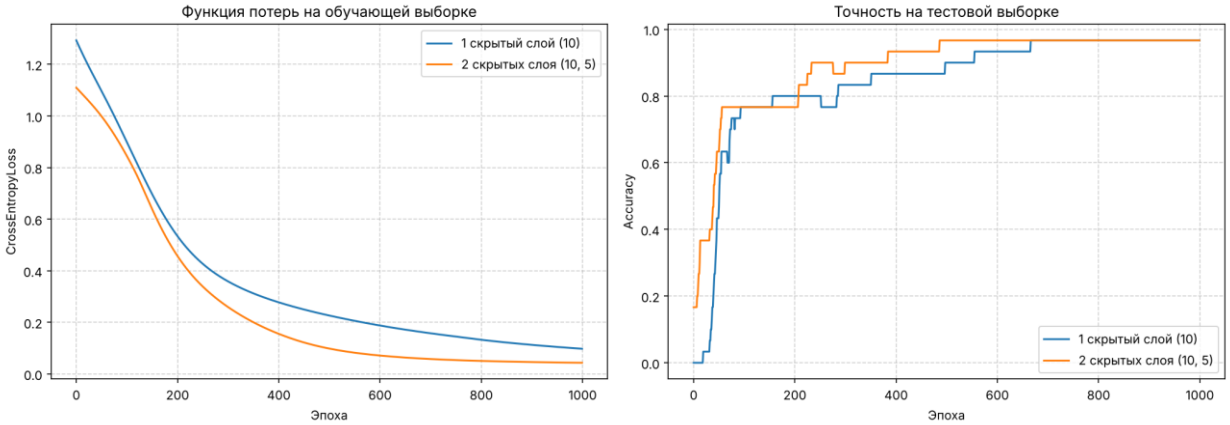


Рисунок 1 - Функция потерь и точность на тестовой выборке по эпохам

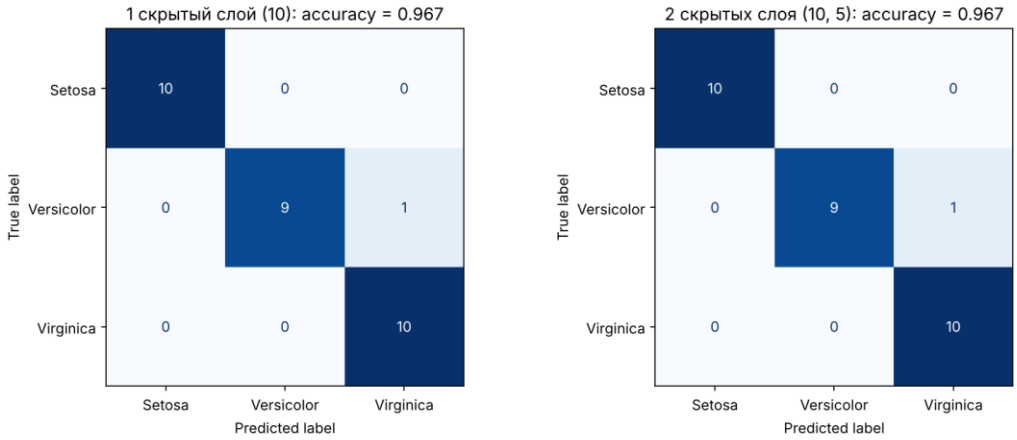


Рисунок 2 - Матрицы ошибок двух моделей

Обе модели достигают accuracy = 0,967 и F1-score (macro) = 0,967 на тестовой выборке (1 ошибка из 30: один ирис Versicolor отнесён к Virginica). Setosa распознаётся безошибочно.

4. Эксперимент: второй скрытый слой

```
Число обучаемых параметров: модель 1 = 83, модель 2 = 123
```

```
Сравнение по 10 запускам (разные seed):
1 скрытый слой (10): accuracy = 0.9633 ± 0.0100, min = 0.9333, max = 0.9667
2 скрытых слоя (10, 5): accuracy = 0.9667 ± 0.0149, min = 0.9333, max = 1.0000
```

Добавление второго скрытого слоя (5 нейронов) ускоряет обучение: функция потерь снижается быстрее и к 1000-й эпохе ниже (0,043 против 0,098), а максимальная точность на тесте достигается примерно к 480-й эпохе вместо 670-й. Однако итоговая точность практически не меняется: по 10 запускам с разной инициализацией весов 0,963 +- 0,010 для одного слоя и 0,967 +- 0,015 для двух. Разница в пределах одного объекта тестовой выборки и не является значимой. Для такой простой задачи достаточно одного скрытого слоя; вторая модель сложнее (123 параметра против 83) и даёт больший разброс результатов.

Вывод: в ходе лабораторной работы я построил, обучил и оценил многослойный перцептрон на PyTorch для классификации ирисов: подготовил данные и преобразовал их в тензоры, описал архитектуру как класс nn.Module, реализовал цикл обучения с функцией потерь CrossEntropyLoss и оптимизатором Adam. Сеть с одним скрытым слоем из 10 нейронов достигла accuracy = 0,967 и F1 = 0,967 на тестовой выборке. Добавление второго скрытого слоя из 5 нейронов ускорило снижение функции потерь, но не повысило итоговую точность заметно (0,967 ± 0,015 против 0,963 ± 0,010), поэтому для датасета Iris достаточно простой архитектуры с одним скрытым слоем.